Inventory API — Documentación de integración con serviciodetercero.com

1. Objetivo y alcance

La Inventory API expone tres funciones de control de inventario a los clientes internos y delega la operación real en el proveedor externo `serviciodetercero.com`. Nuestra API es una **fachada**: autentica al cliente, valida la entrada, llama al tercero, traduce sus respuestas a un contrato propio y estable, y registra cada transacción con un identificador de correlación.

Función	Endpoint propio	Endpoint del tercero
Alta de usuario	`POST /users`	`POST /connect/upload`
Consultar producto	`GET /products/{productId}`	`GET /getProduct`
Cargar producto	`POST /products`	`POST /upload/product`

Fuera de alcance: cualquier otra operación (listar, modificar, borrar), caché de productos, colas o reintentos diferidos. El flujo detallado de cada función, con todas sus transacciones, está en `dss.md`.

2. Componentes

```
Cliente ──HTTPS/JSON──▶ Inventory API ──HTTPS/JSON──▶ serviciodetercero.com
                              │
                              └──▶ Base de datos (solo usuarios)
```

- **Cliente**: se autentica contra nuestra API con un Bearer token. Nunca habla con el tercero ni conoce sus credenciales.
- **Inventory API**: servicio HTTP stateless. Guarda las credenciales del tercero en un gestor de secretos, no en código ni en el repositorio.
- **Base de datos**: persiste usuarios. Los productos no se guardan localmente; el tercero es la fuente de verdad del inventario.
- **serviciodetercero.com**: proveedor externo. Su contrato (métodos, payloads, autenticación) no está especificado en el enunciado; ver sección 7.

3. Contrato de la Inventory API

Todas las respuestas son JSON. Los errores comparten un formato único para que el cliente los maneje de forma uniforme:

```
{ "code": "PRODUCT_RESERVED", "message": "Product exists but is reserved", "upstreamCode": "PD-002",
"correlationId": "4f1c..." }
```

Endpoint	Request	Éxito	Errores propios
`POST /users`	`{ "email", "name" }` (campos mínimos; el tercero puede requerir más)	`201 Created` con el usuario	`400 VALIDATION_ERROR`, `401 UNAUTHORIZED`, `409 USER_ALREADY_EXISTS`, `500 INTERNAL_ERROR`, `502 UPSTREAM_ERROR`, `504 UPSTREAM_TIMEOUT`
`GET /products/{productId}`	path param	`200 OK` con el producto	`400`, `401`, `404 PRODUCT_NOT_FOUND`, `409 PRODUCT_RESERVED`, `409 PRODUCT_OUT_OF_STOCK`, `502`, `504`
`POST /products`	`{ "sku", "name", "quantity" }`	`201 Created` con el producto	`400`, `401`, `502 UPSTREAM_REJECTED`, `502 UPSTREAM_ERROR`, `504`

Toda respuesta, exitosa o no, devuelve el header `X-Correlation-Id`.

4. Mapeo de respuestas del tercero

Esta tabla es el corazón de la integración. Los códigos del tercero son los que define el enunciado; el mapeo a HTTP y a códigos propios es decisión nuestra.

Función	Tercero responde	Nuestra API responde	Por qué
Consultar	`PD-001` no existe	`404 PRODUCT_NOT_FOUND`	Es exactamente la semántica de 404.
Consultar	`PD-002` existe, reservado	`409 PRODUCT_RESERVED`	El recurso existe pero su estado impide la operación: 409 Conflict. No es 404 porque el producto sí existe.
Consultar	`PD-003` existe, agotado	`409 PRODUCT_OUT_OF_STOCK`	Mismo criterio que PD-002. El `code` distingue los casos; el HTTP agrupa la categoría.
Cargar	`200` cargado	`201 Created`	Nuestro cliente creó un recurso; 201 es más preciso que repetir el 200 del tercero.
Cargar	`401` / `403` falló la transacción	`502 UPSTREAM_REJECTED`	El tercero rechazó **nuestras** credenciales o permisos. El cliente no hizo nada mal: devolverle 401 lo induciría a re-autenticarse contra nosotros sin sentido. 502 dice "el upstream falló", que es la verdad. Además se dispara una alerta a operaciones.
Cualquiera	código no documentado, 5xx	`502 UPSTREAM_ERROR`	Fallo del proveedor. Se loguea el código recibido para detectar cambios no anunciados en su contrato. En escrituras, un 5xx deja el resultado **incierto** (ver sección 5).
Cualquiera	sin respuesta	`504 UPSTREAM_TIMEOUT`	Timeout propio, para no dejar al cliente colgado. En un POST el resultado queda **desconocido** (ver sección 5).

El campo `upstreamCode` conserva siempre el código original del tercero. Así nadie tiene que adivinar qué significaba un `409` cuando se investiga un incidente.

Alternativa evaluada para PD-002 y PD-003: responder `200` con un campo `availability: "reserved" | "out_of_stock"`. Es válida si el consumidor principal es una pantalla que muestra estado. Se descartó porque el enunciado los define como códigos de **error** y mantenerlos como errores respeta esa semántica y evita que un cliente trate un producto agotado como disponible por no mirar un campo.

5. Decisiones transversales

Tema	Decisión
Autenticación del cliente	Bearer token en `Authorization`. Se valida antes de cualquier llamada al tercero: una petición no autenticada nunca consume cuota del proveedor.
Credenciales del tercero	Viven en un gestor de secretos y se inyectan por configuración. Un 401/403 del tercero genera alerta porque suele indicar credencial vencida o rotada.
Correlación	La API genera un `X-Correlation-Id` por petición, lo propaga al tercero y lo incluye en logs y respuestas. Es la primera herramienta de diagnóstico en una integración.
Timeouts	Timeout de conexión y de respuesta hacia el tercero (valor orientativo: 5 s). Sin timeout, un proveedor lento agota los recursos de nuestra API.
Reintentos	Solo en `GET /getProduct` (idempotente), un reintento con backoff corto. **Nunca** en los POST: reintentar un alta o una carga puede duplicar registros si el tercero no es idempotente. En escrituras hay que distinguir la respuesta del tercero: un **4xx** es un rechazo de la solicitud, se corrige la causa antes de reintentar; un **5xx** o un **timeout** no prueban que la operación no ocurrió, porque el tercero pudo procesarla y fallar después (por ejemplo al construir la respuesta) o perderse la respuesta. Por eso un 502 por 5xx o un 504 en escritura se loguean para reconciliación y no se reintentan a ciegas.

Tema	Decisión
Orden en el alta de usuario	Primero el tercero, después la base local. Si el tercero responde **4xx**, rechazó la solicitud: no queda nada creado y se corrige la causa antes de reintentar. Si responde **5xx** o **no responde**, el resultado es incierto: pudo haber creado el usuario y fallar después. Aplica la regla de reintentos: log para reconciliación, sin reintento ciego. Si falla el INSERT local después de que el tercero aceptó, queda un usuario en el tercero sin copia local: se loguea como ERROR con `correlationId` para reconciliación manual. Es una ventana pequeña y aceptable para esta primera versión.
Validación de entrada	Esquema estricto antes de salir hacia el tercero. Errores de validación responden 400 sin tocar el proveedor.
Logs	Una línea por transacción con el tercero: endpoint, status recibido, código de negocio, latencia y `correlationId`. Sin payloads completos, para no filtrar datos personales a los logs.
Transporte	`https://` hacia el tercero. El enunciado escribe `http://`; si el proveedor realmente no ofrece TLS, es un riesgo que hay que escalar antes de salir a producción, porque viajan datos de usuarios.

6. Flujos resumidos

1. **Alta de usuario**: validar → `POST /connect/upload` → `INSERT` local → `201`. Ramas de error en `dss.md` §1.
2. **Consultar producto**: validar → `GET /getProduct` → traducir PD-00x → respuesta. `dss.md` §2.
3. **Cargar producto**: validar → `POST /upload/product` → `201` o `502`. `dss.md` §3.

7. Supuestos

Lo que el enunciado no define y tuvimos que asumir. Cada uno debe confirmarse con el proveedor antes de implementar.

Supuesto	Base
El tercero recibe JSON por HTTP y responde JSON.	Práctica habitual; no especificado.
`/connect/upload` y `/upload/product` son `POST`; `/getProduct` es `GET` con el id por query string.	Inferido de los nombres y de la semántica de cada operación.
Los códigos `PD-00x` vienen en el cuerpo de la respuesta, con un status HTTP que puede ser 200 o 4xx.	El enunciado los llama "códigos de error", no códigos HTTP. El cliente HTTP debe leer el cuerpo siempre, no solo el status.
El tercero autentica a nuestra API con alguna credencial (API key o token).	El enunciado menciona 401/403, que implican autenticación.
El tercero **no** garantiza idempotencia en los POST.	Supuesto conservador; justifica no reintentar.
El alta de usuario en el tercero no devuelve un identificador que debamos guardar.	No especificado. Si lo devuelve, se persiste junto al usuario local.
Los campos de usuario y producto son los mínimos razonables (`email`, `name`; `sku`, `name`, `quantity`).	El enunciado no define el modelo. Se ajustarán al contrato real del tercero.

8. Riesgos y evolución

- **Contrato del tercero sin especificar**: es el mayor riesgo. El primer paso real del proyecto es obtener su documentación o un entorno de prueba y validar cada supuesto de la sección 7.
- **Errores 5xx y timeouts en escrituras**: ni un 5xx del tercero ni un timeout prueban que la operación no ocurrió. La mitigación inmediata es el log para reconciliación; la definitiva es acordar idempotencia con el proveedor (clave de idempotencia por petición, o consultar por `email`/`sku` antes de reintentar).
- **Inconsistencia usuario tercero / local**: si la ventana descrita en la sección 5 resulta inaceptable, el siguiente paso es persistir el usuario primero en estado `pending_sync` y sincronizar con reintento diferido (patrón outbox). No se implementa ahora para no agregar infraestructura sin evidencia de que hace falta.
- **Cambios no anunciados en los códigos del tercero**: los logs de `UPSTREAM_ERROR` con el código recibido son la señal temprana.
- **Disponibilidad del proveedor**: si el tercero cae, nuestra API responde 502/504 de forma controlada. Un circuit breaker es una mejora natural cuando haya métricas que lo justifiquen.